

LMA Group Project 2026
Project Proposal for TA Review

Negotiate, Then Solve
Solver-Grounded Negotiation for
Conflict-Aware University Timetabling

Domain	Institute management systems (academic timetabling)
Target users	Academic office, faculty, HoDs, lab in-charges, students
Capabilities	Fine-tuning, RAG, Reasoning, Agentic abilities, Tool calling
Research focus	Solver-grounded negotiation: compromises the LLM may offer come only from solver-derived conflict sets

Team members	[Name 1 – Roll No.] [Name 2 – Roll No.] [Name 3 – Roll No.] [Name 4 – Roll No.]
Assigned TA	[TA name]
Date	26 September 2026

Abstract

University timetables change constantly after publication: faculty report unavailability, labs close for maintenance and students report elective clashes. These requests arrive as unstructured text through a portal, email and messaging apps, and are resolved by hand. Large language models (LLMs) can read such requests but produce timetables that violate hard rules, while constraint solvers ensure validity but cannot read requests, explain infeasibility or negotiate. We study *solver-grounded negotiation*: the solver defines what is feasible, minimal correction sets define the candidate concessions, and a negotiation policy selects among those candidates, rather than letting the LLM invent compromises. Requests are compiled into typed constraints; when OR-Tools CP-SAT finds them infeasible, minimal unsatisfiable subsets (MUS) locate the conflict, minimal correction sets (MCS) define the space of possible concessions, and CP-SAT re-solves each relaxation into a concrete timetable. A negotiation agent offers only these solver-verified options, ranked by a published priority policy and a concession ledger, and explains each one in terms that trace back to the solver’s evidence. A human coordinator approves every change before publication. We evaluate on three separate benchmarks: ITC-2007 and synthetic instances for solver validity, 60 planted-conflict scenarios with stakeholder simulators that hide their true flexibility, and a 600-request parsing corpus, plus a small human pilot. Our primary comparison is against a solver-backed LLM agent that negotiates using LLM-generated, rather than solver-derived, alternatives. We hypothesise that solver-grounded negotiation reaches valid, agreed timetables in fewer rounds and with less disruption than that agent, and with more evenly spread concessions than a solver that imposes a solution without negotiation.

Core research question. Can solver-derived conflict sets constrain an LLM negotiator so that it resolves infeasible timetable requests with fewer negotiation rounds and lower disruption than free-form LLM negotiation, while preserving formal constraint validity?

Every other component (fine-tuned parsing, policy retrieval, the portal) exists to feed this question with typed constraints, or to act on its outcome safely. Section 6 states what is and is not claimed as a contribution.

1 Domain and Target Users

Domain. Institute management systems, specifically *academic course timetabling*: assigning course sessions (lectures, tutorials, practicals) to time slots, rooms, faculty and student sections, and keeping that assignment valid as the semester unfolds.

Scope. One department is enough for a rigorous study (Section 4 of the brief asks for a narrow scope): about 30 faculty, 60 course sections, 20 rooms (including 5 labs), a 5-day week of 8 teaching slots, and a 16-week semester.

Target users and what each needs.

Stakeholder	What they need from the system	Authority
Timetable coordinator (academic office)	One place to see every request, valid proposed changes, clear conflict summaries, one-click approve or reject	Approves and publishes every change
Dean / HoD	Short escalation briefs for policy exceptions and deadlocks	Dean: institute policy (Tier 1); HoD: department
Faculty (incl. visiting)	Submit preferences and unavailability in plain language; consistent, explained outcomes	Own classes only
Lab in-charge, exam cell	Report room outages; protect exam blocks; see affected classes	Rooms; fixed calendar blocks
Students / class representatives	Report clashes (electives, back-to-back labs) and get a tracked response	May raise issues; cannot change anything

Table 1: Stakeholders, needs and authority.

2 Problem Statement

A university timetable is built once per semester but changes weekly. Faculty report unavailability, labs close for maintenance, guest lecturers arrive with fixed windows and students report elective clashes. These requests arrive as unstructured text through different channels: the institute portal, email and messaging applications. The academic office resolves them by hand, and one change often silently breaks another constraint. When two requests cannot both be satisfied, there is no systematic way to decide who should concede, so decisions look arbitrary to those affected. Existing tools either solve a fully specified formal model or chat about requests; we are not aware of one that reads requests, checks every proposed change against a formal model, and negotiates balanced, explained compromises. We need a system that does all three while keeping a human in control of what is published.

3 Why the Problem Matters

- **Validity.** A single double-booked room or faculty clash disrupts every student in the affected sections, often for several weeks.
- **Turnaround.** Manual handling takes days, although most requests have a simple, safe resolution that could be found in seconds.
- **Fairness and trust.** Faculty accept compromises more readily when they can see why a compromise was chosen and that concessions rotate over time rather than always falling on the same people.
- **Accountability.** A published timetable is an institutional commitment. Every change needs a traceable reason, a rule citation where relevant, and a named human approver.
- **Generality.** The same pattern (unstructured requests $\rightarrow$ constraints $\rightarrow$ solver $\rightarrow$ negotiated compromise) applies to exam scheduling, hospital rosters and meeting-room allocation.

4 What Is Missing in Existing Approaches

Table 2 summarises what each family of existing approaches offers and what it misses for this problem.

Approach	Strength	What it misses
Timetabling solvers (ITC-2007 methods, CP/MIP)	Valid, optimised timetables	Need a fully formal model; cannot read requests, explain infeasibility or negotiate
LLM scheduling directly	Reads plain language	Violates hard constraints; no feasibility guarantee
LLM as optimisation modeller (NL4Opt-style)	Turns text into a formal model	One-shot; infeasibility is a failure, not the start of a negotiation
Generic tool-using LLM agents	Multi-step tool use	Invent options that may be infeasible; no priorities, authority checks or fairness memory
Infeasibility explanation in CP (QuickXplain, MUS/MCS)	Finds minimal conflicting sets	Expert-facing output; no plain-language explanation, prioritisation or negotiation

Table 2: Existing approaches and the gap.

The gap. Each ingredient exists separately (Section 5). What has not been studied is their combination as an *interpretation layer* between the solver and the people. We ask whether a layer that combines solver-derived conflicts, explicit relaxation sets, stakeholder negotiation and a persistent concession history resolves conflicts better than LLM-only negotiation and than a solver that does not negotiate.

5 Related Work

University timetabling and minimal-change repair. Course timetabling is a standard benchmark problem; ITC-2007 Track 3 defines the curriculum-based variant we use for solver validity [1, 2]. Repairing a published timetable with as few changes as possible is the *minimal perturbation problem* [10, 11]. These methods assume the new constraints are already formal and consistent; they do not ask anyone to concede.

LLMs as optimisation and constraint modellers. NL4Opt [6], OptiMUS [12] and Chain-of-Experts [13] translate text into LP/MILP models, and “Holy Grail 2.0” [14] argues for the same for constraint programming. They treat the text as one specification to be modelled; infeasibility is an error to fix, not the start of a conversation with the people who wrote the constraints.

LLM agents for planning and scheduling. Benchmarks such as TravelPlanner [15] show that LLM agents regularly violate hard constraints when they plan alone. The LLM-Modulo position [16] argues for pairing LLMs with external verifiers; our system is an instance of this pattern, with CP-SAT as the verifier and generator of options.

Interactive and conversational optimisation. Interactive optimisation keeps a human in the loop to refine the model [17]; conversational constraint modelling does so in natural language [18]. Lawless et al. [19] present MeetMate, which combines an LLM with constraint programming for interactive meeting scheduling with a single user. We extend this to several stakeholders whose requests conflict with each other, where the system must decide whom to ask and in what order.

Explaining infeasibility. QuickXplain [4], MUS and MCS enumeration [5, 20] and step-wise explanations [21] identify why a constraint system is infeasible and what could be relaxed; see [22] for a survey. OptiChat [23] goes further: an interactive natural-language system that uses an optimisation solver to identify the sources of infeasibility and suggest repairs that restore feasibility. These infeasibility-diagnosis systems produce the evidence our negotiator relies on, but they serve a single modeller and do not study multi-stakeholder negotiation over solver-derived relaxations.

Negotiation with language models. Dialogue agents have been trained to negotiate over item splits [24], and LLMs have been evaluated in multi-party negotiation games [25]. In these settings the space of deals is given; in ours it has to be computed from an infeasible constraint model, and a deal is only valid if the solver agrees.

Fairness in allocation. Balancing efficiency against an even distribution of burden is a long-standing question in resource allocation [26]. We do not claim a universally fair policy; we measure whether an explicit, published policy with memory of past concessions spreads burden more evenly than flat weights or free-form negotiation.

6 Research Questions and Hypotheses

RQ1 (core): Solver-grounded negotiation. Does solver-grounded negotiation (compromise options derived from minimal correction sets and instantiated by CP-SAT) resolve more conflicts, with fewer negotiation rounds and less disruption, than free-form LLM negotiation, while every accepted repair remains valid against the formal model?

When a conflict counts as resolved. A scenario is *resolved* when every affected stakeholder has accepted one offered alternative and the combined repair re-solves feasibly. Otherwise it ends as *escalated*: deadlock (every option refused), timeout (no reply within the round limit N), or an option that touches Tier 1–2 and so needs a higher authority. Every system is reported on the same three quantities, so that none can look efficient by escalating hard cases early: the *resolution rate*, the mean and median *rounds conditional on resolution*, and the *escalation rate* (with escalation reasons and precision), all per scenario and using this definition.

- **H1a (validity):** our system has zero hard-constraint violations among the options it proposes, because every candidate is solver-verified, while B3 (and A1) have a higher rate of candidate options the solver rejects, because their alternatives are LLM-generated. Final accepted repairs are checked by CP-SAT in both systems, so validity is compared at the candidate

level; LLM-only scheduling (B1), which has no such check, is expected to have a non-zero final violation rate.

- **H1b (efficiency):** our system achieves a higher or equal resolution rate than B3 and requires fewer negotiation rounds among resolved scenarios.
- **H1c (quality):** our system’s outcomes are closer to the oracle (Section 13.2) than those of B2 and B3, measured by the oracle objective.

Concession burden and its spread across stakeholders (Section 9.7) are reported as *metrics* for RQ1, with the priority-and-ledger ablation (A2) showing their effect. They are not a separate research question.

RQ2: Grounded explanation. Does grounding explanations in solver-derived conflicts improve factual faithfulness and stakeholder acceptance compared with free-form LLM explanations?

- **H2a:** a higher share of explanation claims trace to a conflict-set constraint, a cited rule or a solver result.
- **H2b:** higher first-proposal acceptance by simulated and human stakeholders.

RQ3 (secondary): Efficient parsing. Can a small fine-tuned compiler, behind a lightweight router, handle routine requests with substantially lower latency and fewer LLM calls while preserving constraint-compilation accuracy?

- **H3a:** routing lowers median parsing latency and LLM calls per request compared with sending every request to the System Two LLM.
- **H3b:** compilation F1 on the held-out test partition is not meaningfully lower than LLM-only parsing; we report the difference with a 95% confidence interval.

How hypotheses are tested. Hypotheses are directional. We report effect sizes with 95% bootstrap confidence intervals and paired significance tests (Section 13.6) rather than fixed cut-off values, because no prior work or pilot yet justifies a specific threshold. Results that go against a hypothesis will be reported. For example, if free LLM negotiation matches ours on rounds but fails on validity, that trade-off is itself a finding.

What is and is not claimed. The research contribution is the solver-grounded interpretation layer (L5 in Figure 1). Parsing, retrieval and tool calling are supporting mechanisms, evaluated only as far as the research questions need them. The portal, email and messaging channels, sign-in and the knowledge graph are prototype infrastructure; they are not evaluated as research contributions.

7 Capabilities and Why the Problem Needs Each

All five capabilities are used, and each does work that the problem cannot do without. Every capability has its own metric (Section 13); ablations target only the components the research questions depend on.

Capability	Why the problem needs it	Where it lives	How it is measured
Fine-tuning	Hundreds of routine requests must become constraint JSON quickly and cheaply	QLoRA-tuned compiler (about 2B); calibrated router	Compilation F1, exact match; routing ECE; latency, LLM calls
RAG	Whether a request is allowed depends on institute rules that vary by institute	Policy agent over the handbook	Allow/deny accuracy; citation precision/recall
Reasoning	Infeasibility must be diagnosed and options ranked, with inspectable steps	Conflict analysis, ranking, explainer	MUS accuracy; ranking agreement with oracle; faithfulness
Agentic abilities	A conflict takes many steps across several people, with memory and escalation	Negotiator, concession ledger, escalation	Resolution rate; rounds; gap to oracle; burden spread

Capability	Why the problem needs it	Where it lives	How it is measured
Tool calling	Validity comes from external systems, and some tools change state	CP-SAT, validator, resource DB, publisher behind approval	Valid tool calls; error recovery; unsafe-action rate

Table 3: Capability justification and measurement.

8 Proposed System and Architecture

8.1 Overview

Figure 1 follows one request through the system. Steps ①–④ turn a plain-language message into a checked constraint model, step ⑤ is the research contribution, and step ⑥ keeps a human in control of what is published.

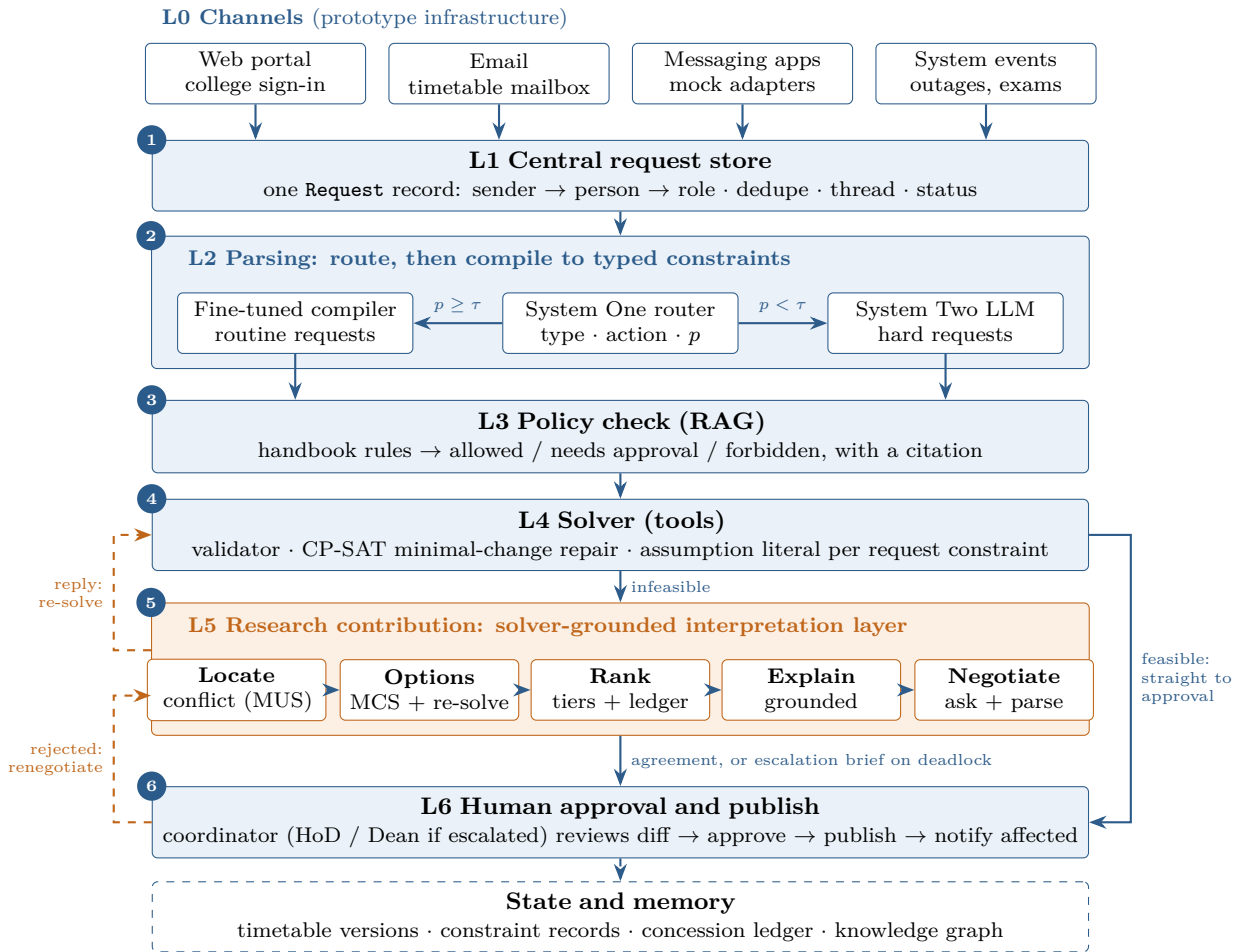


Figure 1: Architecture. Numbered badges match the walkthrough below. Solid arrows show the main flow; dashed orange arrows are feedback loops. The orange layer (L5) is the research contribution.

- ① **Receive (L0–L1)**. Every channel turns a message into one **Request** record carrying the sender’s identity and role. This is infrastructure; experiments inject synthetic requests here.
- ② **Parse (L2)**. The System One router predicts the request type, proposes the next action and returns a confidence p ; a deterministic authority check, not the model, decides whether the sender may make the request. Routine requests ($p \geq \tau$) go to the small fine-tuned compiler; ambiguous or multi-part requests go to the System Two LLM. Both produce the same typed constraint JSON.

- ③ **Check policy (L3).** The policy agent retrieves the relevant institute rules and returns *allowed*, *needs approval* or *forbidden* with a citation. A forbidden request is answered with a compliant alternative and never reaches the solver.
- ④ **Solve (L4).** The validator resolves names to IDs and checks the schema; CP-SAT repairs the published timetable with as few changes as possible. If the result is *feasible*, it goes straight to approval (right-hand path).
- ⑤ **Resolve the conflict (L5).** If it is *infeasible*, the interpretation layer takes over: it locates the conflict (MUS), builds options by relaxing each MCS and re-solving with CP-SAT, ranks them by tier, disruption and the concession ledger, writes a grounded explanation and asks the right stakeholder. Each reply updates the constraints and is re-solved (dashed loop) until there is agreement or the case is escalated.
- ⑥ **Approve and publish (L6).** The coordinator sees the diff, the affected people, the conflict history and the fairness report, then approves or rejects. Only approval publishes a new, reversible version and notifies the people affected; a rejection returns the case to step ⑤.

8.2 Architectural invariant: the LLM cannot change state

No model output can publish, move a class or relax a constraint directly. Every LLM output is a *structured proposal* that passes through a fixed chain of deterministic checks before any state-changing tool runs:



Figure 2: The only path from model output to a change in state.

The orchestrator, not the agent, holds the publisher and the timetable-writing tools; they are never exposed to a model as callable tools. A failure at any step returns the case to negotiation or to a human.

8.3 Request lifecycle

Every request moves through a fixed set of states. The state machine makes the agent’s behaviour auditable and gives the evaluation clear checkpoints.

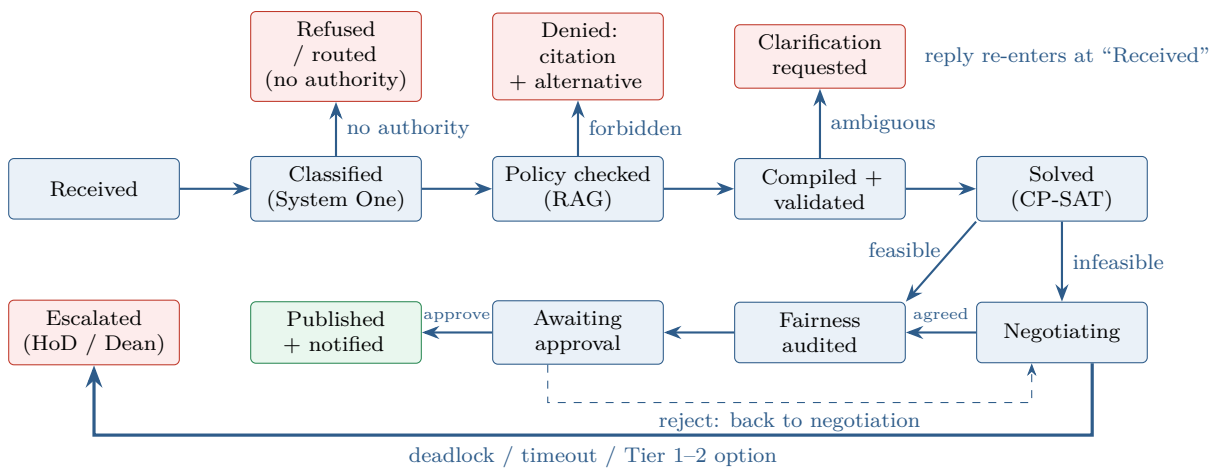


Figure 3: Request lifecycle. Red states are safe exits; nothing reaches “Published” without human approval.

8.4 Core data object: the constraint record

Everything the solver and negotiator act on is a typed constraint record carrying its owner, tier, source and validity window; the private justification never leaves the record:

```
{ "id": "C-0412", "type": "unavailable", "hard": true, "tier": 3,
  "owner": "F-102", "when": {"days": ["Tue","Wed","Thu"], "weeks": [7]},
  "justification": "verified", // none | stated | verified; kept private
  "source": {"request": "R-2291"} }
```

9 Methodology

9.1 Ingestion and identity (prototype infrastructure)

Users sign in with their college email (OAuth 2.0 / OpenID Connect), and their role comes from the staff and student directory; the sender’s identity decides what they may change. Text inside messages is always treated as data, never as instructions. Experiments bypass this layer and feed synthetic requests directly into the request store.

9.2 Hybrid System One / System Two parsing

Parsing uses two paths chosen per request:

1. The **System One router**, a calibrated lightweight classifier, returns a probability distribution over request types (*preference, unavailability, swap, room issue, clash report, policy question, out-of-scope*), a proposed next action and a confidence p . We treat the specific classifier as replaceable: the research question is whether routing helps, not which router is best. Whether the sender is permitted to make the request is enforced separately by a deterministic authority check over the directory and the *has_authority_over* relation; the router’s output cannot override it.
2. If $p \geq \tau$ and the type matches a known template, the **QLoRA compiler** produces the constraint JSON directly.
3. Otherwise the request goes to the **System Two LLM**, which decomposes it, asks a clarifying question if needed, and produces constraints.
4. A deterministic **validator** checks the schema and resolves names to IDs (“Dr. Rao” $\rightarrow$ F-102). Failures return to the requester as a clarification question.

The threshold τ is chosen on the validation split to maximise the System-One share subject to a compilation-F1 tolerance fixed before the test split is used (RQ3). The tolerance is an engineering setting, not a hypothesis.

9.3 Constraint model

Let S be course sessions, T time slots, R rooms, F faculty and G student groups (sections). Decision variables $x_{s,t,r} \in \{0, 1\}$ assign session s to slot t in room r .

$$\sum_{t \in T} \sum_{r \in R} x_{s,t,r} = 1 \quad \forall s \in S \quad (\text{every session scheduled once}) \quad (1)$$

$$\sum_{s \in S} x_{s,t,r} \leq 1 \quad \forall t, r \quad (\text{no room double-booking}) \quad (2)$$

$$\sum_{s \in S(f)} \sum_r x_{s,t,r} \leq 1 \quad \forall f \in F, t \quad (\text{no faculty clash}) \quad (3)$$

$$\sum_{s \in S(g)} \sum_r x_{s,t,r} \leq 1 \quad \forall g \in G, t \quad (\text{no student-group clash}) \quad (4)$$

$$x_{s,t,r} = 0 \quad \text{if } \text{size}(s) > \text{cap}(r) \text{ or room type mismatches} \quad (5)$$

Institute rules (maximum consecutive hours, lunch break, contact hours per course) are added as further linear constraints. Every request-derived constraint C_k is guarded by an assumption literal a_k so that it can be switched on or off:

$$a_k \Rightarrow C_k.$$

In **repair mode** the objective adds a disruption penalty $\sum_{s,t,r} |x_{s,t,r} - x_{s,t,r}^0|$ relative to the published timetable x^0 , so changes stay local.

9.4 Priority model: tiers first, then scores

Priorities are lexicographic across tiers and weighted within a tier. No number of preferences can outweigh a rule.

Tier	Constraint class	Who may relax it	Examples
T0	Physical impossibilities	Nobody	No double-booking; room capacity
T1	Institute policy	Dean / academic council	Maximum 3 consecutive hours; common lunch break
T2	Fixed institutional commitments	Academic office	Exam blocks; guest faculty windows
T3	Verified hard unavailability	Owner, with an alternative	Conference, medical leave, clinical duty
T4	Operational requirements	Owner or HoD	Lab needs GPUs or routers; room type
T5	Preferences	Owner, freely	“No classes before 10”; fewer gaps

Table 4: Priority tiers.

Within a tier, each constraint k receives a score that the academic office configures and publishes:

$$\pi_k = w_1 \text{auth}(k) + w_2 \text{impact}(k) + w_3 \text{just}(k) + w_4 \text{lead}(k) + w_5 \text{credit}(k) - w_6 \text{disrupt}(k) \quad (6)$$

where *auth* is the owner’s role authority, *impact* the number of people affected, *just* the justification strength, a structured evidence category set during parsing and policy validation (*none*; *stated*, a reason is given; *verified*, the office has confirmed it), *lead* how early the request arrived, *credit* the owner’s fairness credit from the concession ledger, and *disrupt* the number of classes the constraint forces to move. **A higher π_k means a greater cost of relaxing constraint k .** So a stakeholder who has conceded often (high credit) is more expensive to ask again, and a constraint that forces many other classes to move (high disrupt) is cheaper to relax.

The weights are a policy, not a finding. Whether an early request should outweigh one that affects more students is a normative choice with no universally correct answer. The tiers and weights $w_1, \dots, w_6$ are therefore an experimentally controlled policy configuration that the academic office publishes; we do not claim they are objectively fair. The experiments test whether an *explicit, published and persistent* policy spreads concession burden more evenly than flat weights (ablation A2) or free-form negotiation (B3).

The solver runs in **stages**: satisfy T0–T2 as hard constraints; then maximise the satisfied T3 score and fix that optimum; then T4; then T5 together with the disruption penalty. This guarantees tier order and makes each decision explainable (“your preference lost to a Tier 3 unavailability”).

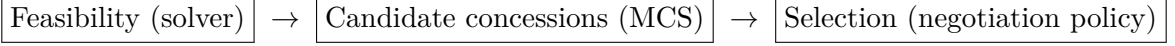
9.5 Conflict analysis

When a stage is infeasible, CP-SAT reports a subset of assumption literals that is sufficient for infeasibility. We reduce it to a **minimal unsatisfiable subset (MUS)** by deletion-based shrinking. We then enumerate a small number of **minimal correction sets (MCS)**: minimal sets of constraints whose removal restores feasibility (by hitting-set duality, every MCS intersects every MUS [5]).

An MCS says *which* constraints must give way, not what the resulting timetable is. Each option is therefore produced in two steps:

$$\text{MUS} \rightarrow \text{MCS } M \rightarrow \text{relax } M \rightarrow \text{CP-SAT re-solve (repair)} \rightarrow \text{timetable } x_M.$$

MCS enumeration generates the space of negotiable concessions; CP-SAT instantiates each concession as an actual timetable. The mechanism therefore has three levels, which are worth keeping apart:



The solver-feasible space holds every timetable that satisfies the formal constraints; the candidate space is the much smaller set of minimal relaxations the enumeration produces, each instantiated as one of those timetables; and the negotiation policy (priorities, ledger, who is asked first) chooses among the candidates with the stakeholders. The LLM never adds a candidate; it explains candidates and parses replies. The candidate space is not guaranteed to contain every useful negotiation: for tractability we cap MCS enumeration at the K cheapest candidates per round ($K = 5$), so the experiment evaluates negotiation over this bounded candidate space. Unless otherwise stated, K is fixed before evaluation and is not tuned on the test scenarios. Where a stakeholder’s concession is partial (“Thursday afternoon instead”), the constraint is replaced by its weaker version rather than removed, and the re-solve is repeated. An option is offered only if its re-solve succeeds, so every proposed compromise is verified feasible against the formal timetable model before any stakeholder sees it. This guarantee is only as good as the model: a request compiled incorrectly (“Lab 2 *requires* routers” parsed as a preference) yields a valid solution to the wrong problem, which is why compilation accuracy is measured separately (RQ3) and a human approves every change.

Each conflict is classified, because the class decides the strategy:

Conflict type	Example	Default strategy
Resource contention	Two labs need Lab 2 on Tuesday afternoon	Find an equivalent resource; otherwise negotiate
Request vs policy	“All 4 hours back-to-back”	Deny with citation and offer a compliant alternative
Capacity shortfall	Lab outage leaves one slot short	Escalate: every option affects students
Cascade	Fixing A breaks B	Solve jointly; show the whole chain
Preference trade-off	Feasible, but someone loses a preference	Decide by score, notify, allow appeal

Table 5: Conflict classes and default strategies.

9.6 Negotiation: the resolution ladder

The negotiator always tries the cheapest resolution first:

Step 1. Auto-substitute. If an equivalent room or slot exists and the change is provably non-disruptive (every hard constraint still holds and no one else’s placement changes), mark the case auto-resolvable: no stakeholder has to be asked. Publication still follows the configured approval policy (Section 8.2).

Step 2. Auto-relax. If some MCS contains only Tier 5 preferences, drop the lowest-scored preference, notify the owner, and allow an appeal.

Step 3. Negotiate. Rank MCS options by cost:

$$\text{cost}(M) = \sum_{k \in M} \pi_k + \lambda_1 \text{disruption}(x_M) + \lambda_2 \Delta D(M)$$

where $\Delta D(M)$ is the increase in the dispersion of concession burden that accepting M would cause (Section 9.7). Ask the owner of the cheapest option first; among equal costs, ask the stakeholder with the *lowest* fairness credit (the one who has conceded least recently). Each message contains a contrastive explanation and two or three concrete, solver-verified alternatives. Replies in plain language are parsed by System Two, constraints are updated, and CP-SAT re-solves. Limits: at most N rounds per conflict and a response deadline, after which the published default policy applies.

Step 4. Escalate. On deadlock, timeout, or any option touching Tier 1–2, follow the reporting chain (Faculty → HoD → Dean) with a one-paragraph brief: the conflict, the options, their costs and the history.

Hidden flexibility. Stakeholders often state a preference as a requirement. The negotiator asks targeted questions (“Would Thursday afternoon work?”) before treating a constraint as absolute; this is what makes negotiation a multi-step agentic task rather than a one-shot answer.

Concession ledger. Each accepted concession adds credit to the conceding stakeholder, lowering their chance of being asked next time. Credit decays each semester so history matters but does not dominate.

9.7 Concession burden and fairness

Concessions are not equal: giving up “Tuesday 2 pm” is not the same as giving up one’s only available teaching slot. Counting concessions would make a fairness score misleading, so each concession is weighted. For stakeholder i ,

$$c_i = \sum_{k \in K_i} \text{importance}(k) \times \text{magnitude}(k),$$

where K_i is the set of i ’s constraints relaxed or weakened, $\text{importance}(k)$ grows with the constraint’s tier and justification (a verified Tier 3 unavailability weighs more than a Tier 5 preference), and $\text{magnitude}(k) \in (0, 1]$ is the share of the original constraint given up (1 if dropped, less if replaced by a nearby alternative). The ledger stores c_i contributions, with the semester decay above.

We report two separate quantities:

- **Concession burden:** the vector $(c_1, \dots, c_n)$ and its total, i.e. how much was conceded in all and by whom.
- **Burden dispersion D :** how unevenly that burden falls. We use the Gini coefficient of $(c_1, \dots, c_n)$ as the main summary, and also report the maximum individual burden and the coefficient of variation, so the conclusion does not depend on one index.

9.8 Explanation

Every explanation is **contrastive** (why this option rather than the requested one) and **grounded**: each claim must map to a constraint in the MUS, a cited rule, or a solver result. Explanations are generated from a structured template filled by the LLM, then checked by a verifier that rejects unsupported claims. Explanations name the *constraint*, never the private *reason*: “Dr. Menon is unavailable”, never the medical detail.

“Your request for Lab 2 on Tuesday afternoon could not be met. **Why:** Dr. Das’s networking lab also needs Lab 2 then (Tier 4, equipment), and Lab 2 is the only room with routers. **Options:** (a) Thursday afternoon in Lab 2, where the GPU machines are confirmed free; (b) Tuesday afternoon in Lab 4, without GPUs. Dr. Das conceded a slot last semester.”

9.9 Relationships used by the interpretation layer

Interpretation decisions are queries over a small stakeholder–constraint graph (prototype infrastructure, stored as NetworkX over the database): *owns* (whom to ask to relax a constraint), *has_authority_over* (whether a request is valid), *reports_to* (the escalation path), *derived_from* (provenance for explanations and audit), *conflicts_with* (filled from MUS results), *affects* (who is notified; disruption cost) and *balance* (fairness credit, which orders who is asked).

10 Use-Case Scenarios by Stakeholder

The evaluation focuses on three core conflict classes from Table 5: **resource contention** (UC3, including deadlocked and Tier 3-versus-Tier 4 variants), **request versus policy** (UC4) and **capacity shortfall** (UC5), with feasible preference trade-offs as a control. The other scenarios are demonstrations of the full system and are not needed to answer the research questions; UC8 is covered by the safety set.

UC	Stakeholder: request	How the system handles it	Role
UC1	Faculty: “no classes before 10 on Mon/Wed”	Tier 5 preference via the fast path; minimal repair; fairness check avoids loading early slots onto one colleague	Demonstration (RQ3 path)
UC2	Faculty: conference in week 7	Tier 3 unavailability for one week; RAG adds the make-up rule; CP-SAT places make-up sessions, else the HoD decides	Demonstration
UC3	Two faculty: same lab, same afternoon	Full interpretation layer (below)	Core (RQ1, RQ2)
UC4	Faculty: all 4 lecture hours back-to-back	Policy agent denies with a citation and offers compliant 2+2 alternatives	Core (policy)
UC5	Lab in-charge: lab closed in weeks 5–6	Bulk repair leaves one lab slot short; every option affects students, so the case escalates	Core (capacity)
UC6	Class rep: elective clash across departments	Student has no authority; the system verifies enrolment and sends solver-checked options to both HoDs	Demonstration
UC7	Office: full semester build	Bulk compilation, parallel negotiation of independent conflict groups, one consolidated diff	Demonstration
UC8	Student email containing an injection	No authority and injection flagged; nothing reaches the solver or publisher	Safety set

Table 6: Use-case scenarios and their role in the evaluation.

UC3 — Two faculty: contention for the same lab (core research scenario)

Stakeholders: Dr. Khan and Dr. Das, faculty. **Channel:** portal and messaging app.

Problem: Both need Lab 2 on Tuesday afternoon; Dr. Khan for GPUs, Dr. Das because only Lab 2 has routers.

How the system helps:

1. Both constraints are Tier 4; the stage is infeasible.
2. Conflict analysis returns the MUS {Khan→Lab 2 Tue PM, Das→Lab 2 Tue PM, routers only in Lab 2} and two MCS options.
3. The resolution ladder finds no equivalent room. Dr. Das conceded last semester (higher credit), so Dr. Khan is asked first: “Would Thursday afternoon in Lab 2 work? GPUs are confirmed free.”
4. Dr. Khan agrees in plain text; System Two parses the reply; CP-SAT re-solves; the ledger records the concession.
5. If neither agrees within the deadline, the HoD receives a one-paragraph brief.

Outcome: agreement in one round with a feasible, policy-consistent and explained option.

Evaluates: RQ1 and RQ2 (MUS/MCS options, ordering, ledger, grounded explanation).

11 Tech Stack

The row marked $\dagger$ is prototype infrastructure and is not evaluated as a research contribution.

Concern	Choice	Why
Core	Python 3.11+, FastAPI + Pydantic, SQLite; environment locked with uv	Typed schemas end to end; one-command, reproducible setup
Solver	Google OR-Tools CP-SAT	Assumption literals give conflict extraction (MUS/MCS)
System One router	scikit-learn: TF-IDF + calibrated logistic regression	Millisecond, calibrated decisions on a CPU; open System-One models [27, 28] are optional drop-ins
Constraint compiler	Qwen-family model (about 2B), QLoRA via PEFT/TRL; served with llama.cpp	Trains on a free 16 GB GPU; runs on a laptop
System Two LLM	Local fine-tuned model (Tier 1); Gemini Flash / Flash-Lite free tier (Tier 2)	Core results do not depend on an external API
RAG	BM25 over rule text + fastembed (bge-small) embeddings	One handbook; lexical matching handles rule numbers and wording
Orchestration	Hand-written state machine	Matches the request lifecycle; easy to log and replay
Evaluation	pytest, pandas, SciPy (bootstrap, McNemar, Wilcoxon)	Reproducible runs with logged traces
Prototype infrastructure $\dagger$	React + TypeScript portal; college OAuth sign-in; email and messaging adapters (mocked in experiments); NetworkX knowledge graph	Makes the system usable as a demonstration

Table 7: Tech stack decisions.

12 Expected Novelty

We study whether an interpretation layer combining solver-derived conflicts, explicit relaxation sets, stakeholder negotiation and a persistent concession history improves conflict resolution over LLM-only and non-negotiating solver baselines. The expected contributions are:

1. **Solver-grounded negotiation via MUS/MCS.** MCS enumeration defines the concessions the agent may offer and CP-SAT turns each into a concrete timetable, so every option is verified feasible against the formal model before a stakeholder sees it. Prior LLM agents propose options they cannot verify [15]; prior infeasibility-diagnosis systems can identify causes of infeasibility and suggest repairs, but do not study multi-stakeholder negotiation over solver-derived relaxations [22, 23].
2. **Persistent, concession-aware negotiation policy.** A published tiered priority policy with a concession ledger, evaluated by weighted concession burden and its spread across stakeholders and semesters.
3. **Evidence-grounded, privacy-preserving explanations.** Explanations checked claim by claim against the conflict set, with a leakage metric for private reasons.

Supporting contributions. A measured latency and LLM-usage study of routing routine requests to a small fine-tuned compiler (RQ3), and an open benchmark of timetabling instances paired with plain-language requests, hidden-flexibility stakeholder simulators and oracle outcomes.

13 Dataset and Evaluation Environment

13.1 Three benchmarks

ITC-2007 establishes solver validity at realistic scale, but it contains no natural-language requests, stakeholders or hidden flexibility. The evaluation is therefore split into three benchmarks, each answering a different question.

Benchmark	Source and construction	Size	Answers
A. Solver validity	ITC-2007 Track 3 (curriculum-based course timetabling) instances with known best solutions, plus a synthetic generator shaped like our department	10 ITC + 10 synthetic	Model correctness, repair, scale
B. Negotiation	Synthetic department instances with planted conflicts in the three core classes; scripted stakeholder simulators with hidden true flexibility and preferred alternatives; oracle outcome per scenario (Section 13.2)	60 scenarios	RQ1, RQ2
C. Parsing	Plain-language requests generated from known target constraints, then paraphrased, made ambiguous, rule-breaking, unauthorised or adversarial; 100 written by team members and checked by two annotators	600 (400 train / 50 val / 150 test)	RQ3
Regulations corpus	Our institute’s academic handbook and timetable rules (public document)	1 handbook	Policy agent
Safety set	Authority violations, prompt injections in email bodies, requests to reveal private reasons	50 cases	Safety
Human pilot	5–10 real faculty rate explanations and negotiation messages; also validates the LLM judge	about 100 ratings	RQ2, judge

Table 8: Benchmarks and supporting data.

Ground truth is known by construction: each request is generated from its target constraint, each conflict is planted, and each oracle outcome is computed by the solver.

Stakeholder simulators. A simulator must not be built to favour our negotiator. Each simulated stakeholder has a hidden profile: the alternative times that really work, whether a room without the requested equipment would do, whether they volunteer an alternative, and whether they reply at all. Profiles are sampled per scenario by a seeded generator that is written before, and independently of, the systems being compared; no system sees the profiles, and all systems face the same stakeholders. We use four policy families:

- S1 Strict:** accepts only its stated constraint or a narrow set of alternatives; rarely volunteers one.
- S2 Flexible:** accepts a wide set of alternatives and often volunteers one when no offer fits.
- S3 Cost-sensitive:** accepts an alternative only if the change it asks for is small (a nearby slot, the same day).
- S4 History-sensitive:** less willing to concede when it has already conceded recently, independent of our ledger.

Responses are stochastic: an acceptable offer is refused, and a message goes unanswered, each with a small probability, so a system cannot succeed by exploiting a deterministic reply rule. The main results are reported per family and pooled, and a conclusion is only claimed if it holds across families.

Preventing train/test leakage in Benchmark C. Requests are generated from templates and then paraphrased, so a random split would put paraphrases of the same template in both train and test, and the fine-tuned compiler could score well by memorising templates. Splits are therefore made by *underlying template and scenario*, never by paraphrase: no template, and no semantically equivalent request, appears in more than one split. The 150-request test partition is further tagged so results can be reported per slice:

- unseen wording (held-out templates for known constraint types);
- unseen combinations of constraints within one request;
- ambiguous wording that requires a clarification question;
- adversarial wording (injections, requests beyond the sender’s authority);
- multi-constraint requests.

13.2 Oracle definition

Each negotiation scenario hides, for every stakeholder, the true set of alternatives they would accept. The oracle (B4) is given these hidden sets; the negotiating agent is not.

What the oracle optimises. The oracle uses exactly the same published policy and objective as our system, so that “distance to the oracle” measures the cost of *not knowing* stakeholders’ flexibility and nothing else. Over every combination A of acceptable alternatives (one per affected stakeholder, including “keep”), it re-solves the timetable with CP-SAT and minimises, lexicographically,

$$J(A) = \left(V_{\text{tier}}(A), \sum_{k \in M_A} \pi_k + \lambda_1 \text{disruption}(x_A) + \lambda_2 D(A) \right),$$

where $V_{\text{tier}}(A)$ is the vector of violated constraints per tier (T0 first; any T0–T2 violation makes A inadmissible), M_A is the set of constraints relaxed or weakened, x_A is the resulting timetable and $D(A)$ is the burden dispersion after the concessions in A . The weights π , λ_1 and λ_2 are the published configuration of Section 9.7; ties are broken by fewer stakeholders asked. If no admissible combination exists, the oracle outcome is *escalate*.

Distance to the oracle. For a scenario resolved by a system with outcome A_{sys} , we report the gap $J(A_{\text{sys}}) - J(A^*)$ on the second component, plus *status agreement*: whether the system reached the same status as the oracle (feasible without negotiation, agreement, or escalation). A system that escalates when the oracle found an agreement, or agrees when the oracle escalates, is counted as a status error rather than given a numerical gap.

13.3 Baselines

B3 is the primary baseline: it is the only other system that negotiates with an LLM, so the comparison with B3 answers RQ1. B1 and B2 are controls that locate the gain: B1 measures how far unconstrained LLM scheduling is from feasible, and B2 measures what negotiation adds over the same optimiser acting alone.

B1 LLM-only (control): the System Two LLM receives the instance and requests and writes the timetable directly. Establishes the feasibility gap of unconstrained LLM scheduling.

B2 Solver, no negotiation (control): the same constraint compilation, tiers, weights, disruption objective and solver as our system, but it directly selects the best feasible repair using only the stated constraints and imposes it without asking anyone. Any difference from our system is therefore due to negotiation, not to the objective.

B3 LLM + solver, free negotiation (primary): the same parsing, solver and stakeholders, but on infeasibility the LLM negotiates with options it invents (no MUS/MCS, no tiers, no ledger); its final agreement is still checked by CP-SAT.

B4 Oracle / upper bound: the same objective as our system with all hidden flexibility known (Section 13.2).

Ablation A1 is the controlled version of the B3 comparison: it keeps tiers and the ledger and replaces only the MCS options with LLM-invented ones. Each question the evaluation asks therefore has one primary comparison:

Question	Primary comparison
Does solver-grounding help negotiation? (RQ1)	Our system vs B3
Does negotiation itself help?	Our system vs B2
How far is unconstrained LLM scheduling from feasible?	B1
Do MCS-derived candidates specifically matter?	Our system vs A1
Does the persistent priority and concession policy matter?	Our system vs A2
Do grounded explanations matter? (RQ2)	Our system vs A3
Does System-One routing matter? (RQ3)	Our system vs A4

Table 9: Evaluation questions and their primary comparisons.

Ablation	What it tests	RQ
A1: MCS options → LLM-invented options	Value of solver-grounded options	RQ1
A2: Tiers + ledger → flat weights, no ledger	Value of the priority model and fairness memory	RQ1
A3: Grounded → free explanations	Faithfulness and acceptance	RQ2
A4: System One routing → LLM for all parsing	Latency and LLM usage	RQ3

Table 10: Ablation plan: one ablation per core claim.

13.4 Ablations

The ablation study is deliberately small: each ablation removes exactly one conceptual component that a research question depends on. A2 removes tiers and the ledger together because they form one component, the persistent priority and concession policy. Other components (fairness critic, escalation step, RAG) are evaluated through their own metrics in Table 11 rather than through separate ablations. The QLoRA compiler is compared with the prompted System Two model as part of the parsing evaluation (H3b), not as a separate ablation.

13.5 Metrics

Area	Metrics
Outcome	Final repair violation rate; soft-preference satisfaction; classes moved per repair; gap to oracle objective and status agreement
Candidate validity	Invalid candidate rate (share of offered options the solver rejects); solver rejection rate of accepted options; invalid candidates per resolved scenario
Fine-tuning	Constraint-compilation F1 and exact match; per-field accuracy
System One layer	Routing accuracy; calibration (ECE, Brier); share of requests on fast path; p50/p95 latency; LLM calls per 1,000 requests
RAG	Allow/deny accuracy; rule-citation precision and recall
Reasoning	MUS identification accuracy; option-ranking agreement with oracle (Kendall’s τ); explanation faithfulness (% claims traced)
Agentic	Resolution rate; rounds conditional on resolution (mean, median); escalation rate, reasons, precision and recall
Fairness	Weighted concession burden per stakeholder; burden dispersion (Gini, maximum individual burden, coefficient of variation) across stakeholders and semesters
Tool calling	Valid tool calls (schema and arguments); error-recovery rate
Safety	Unapproved-publish rate (target 0); authority-violation rate; tier-violation rate; private-reason leakage rate; injection success rate
Human	Explanation clarity and acceptability (Likert); first-proposal acceptance

Table 11: Metrics, including capability-specific and safety metrics.

13.6 Protocol

Experiments run in three tiers, so that the core results never depend on the availability of a free API tier:

Tier 1 All core experiments (Benchmarks A–C, B1–B4, A1–A4) with the local fine-tuned model and cached responses. These are the results the conclusions rest on.

Tier 2 Stronger-model validation: the main comparison repeated with Gemini Flash on a stratified subset of scenarios, to check that the conclusions hold with a stronger System Two model.

Tier 3 Human pilot (explanations and negotiation messages).

- 3 seeds for the main comparison (our system against B1–B3); ablations run once on a stratified subset of 30 scenarios.
- Effect sizes with 95% bootstrap confidence intervals; McNemar’s test for paired pass/fail outcomes and Wilcoxon signed-rank for paired continuous outcomes.
- Any LLM judge (for example, explanation clarity) is validated against two human annotators on at least 100 items; we report Cohen’s κ .
- Intermediate steps are logged and analysed: parse output, retrieved rules, MUS, options offered, messages sent, replies, final decision.
- Qualitative analysis of difficult cases and a failure taxonomy with counts (Section 7 of the brief).

Results structure. The results are organised around three primary outcomes, each per configuration, pooled and per stakeholder family: (1) resolution rate, (2) rounds per resolved scenario, and (3) disruption and the gap to the oracle objective. Three secondary outcomes follow: (4) invalid candidate rate, (5) concession burden dispersion, and (6) explanation faithfulness.

14 Data and Safety Plan

- **No real personal data in experiments.** All experimental data are synthetic or from public benchmarks (ITC-2007). The human pilot uses consenting volunteers and records only ratings, not personal details.
- **Human approval gate.** Nothing is published without the coordinator’s explicit approval. Target unapproved-publish rate: 0.
- **Authority enforcement.** The `has_authority_over` relation is checked deterministically before any constraint is created; the LLM cannot override it.
- **No silent relaxation.** Tier 0–2 constraints are never relaxed without a named human with the right authority.
- **The LLM cannot change state.** This is an architectural invariant (Section 8.2), not a prompt instruction: model output is only ever a structured proposal, and state-changing tools are held by the orchestrator and run only after validation, policy and authority checks, CP-SAT verification and human approval.
- **Prompt-injection defence.** Message content is treated as data; because of the invariant above, a successful injection can at worst produce a proposal that the deterministic checks or the coordinator reject.
- **Privacy.** Justifications (for example, medical reasons) are stored with restricted access and never appear in explanations sent to others; leakage is measured.

- **Third-party API use.** Prompts sent to the free tier of the Gemini API may be used by Google to improve its products, so only synthetic and public benchmark data are ever sent to it. Human-pilot ratings stay on team machines.
- **Sandbox.** The prototype runs against a copy of the timetable database; no production system is touched.
- **Transparency.** Priority weights and tier rules are published to all users.

15 Compute Plan

Item	Estimate	Resource
QLoRA constraint compiler	About 2B model, 400 training examples, about 2–4 GPU-hours per run $\times$ 3 runs (seeds) + 2 hyperparameter runs	Free Colab / Kaggle T4 (16 GB): about 15 GPU-hours
System One router	TF-IDF + logistic regression; trains in seconds	Laptop CPU
CP-SAT solver	Department-size instances solve in seconds to minutes; about 3,000 solves in total	Laptop CPU
<i>Tier 1: core experiments, local model with response cache</i>		
Parsing experiments	150 test requests $\times$ 2 configurations $\times$ 3 seeds $\approx$ 900 calls	Local model (llama.cpp)
Negotiation: main comparison	60 scenarios $\times$ about 15 calls $\times$ 3 configurations (ours, B1, B3) $\times$ 3 seeds $\approx$ 8,100 calls	Local model
Negotiation: ablations	30 scenarios $\times$ about 15 calls $\times$ 3 ablations (A1–A3) $\times$ 1 seed $\approx$ 1,350 calls	Local model
Explanations and safety set	about 1,000 calls	Local model
<i>Tier 2: validation with a stronger API model</i>		
Parsing, System Two reference	150 test requests $\times$ 1 run $\approx$ 150 calls	Gemini Flash
Negotiation subset	20 scenarios $\times$ about 15 calls $\times$ 3 configurations $\times$ 1 seed $\approx$ 900 calls	Gemini Flash + Flash-Lite
LLM judge	about 1,000 calls, validated against human annotators	Gemini Flash-Lite
Total API usage	about 2,000 calls, about 4–5 million tokens (mostly input)	Gemini API free tier

Table 12: Compute budget.

The scientific plan does not depend on a free API tier. All core (Tier 1) experiments run on the locally served fine-tuned model, with every response cached by prompt hash so reruns are free and reproducible. Google’s Gemini Flash models, through the free tier of the Gemini API, are used only for Tier 2 validation on a subset and for the LLM judge: Gemini Flash for the System Two agent, and the lighter Flash-Lite for the stakeholder simulators and the judge. No paid API spend is planned. Tier 2 runs go through a rate-limited queue, and quotas are checked against Google’s published rate limits before each run, because they change over time. If the free tier shrinks or disappears, Tier 2 is reduced or dropped and the conclusions still stand on Tier 1.

16 Timeline and Team Roles

Roles (to be confirmed): Member 1 — solver and conflict analysis; Member 2 — parsing, fine-tuning and System One layer; Member 3 — agents, negotiation and RAG; Member 4 — evaluation harness, simulators, portal and write-up.

Weeks	Milestone
1–2	TA approval; related-work review; data model and constraint schema frozen
3–4	Instance generator, request corpus and annotation; CP-SAT model with assumption literals
5–6	Parsing layer (System One, QLoRA compiler, System Two); policy RAG; validator
7–8	Interpretation layer: tiers, MUS/MCS, resolution ladder, ledger, explainer
9	Stakeholder simulators; portal and email ingestion (minimal)
10–11	Experiments, baselines, ablations; human pilot
12	Analysis, failure taxonomy, write-up and open-source release

Table 13: Twelve-week plan.

17 Expected Risks and Fallback Plan

Risk	Fallback
Stakeholder simulators seem artificial or favour our system	Four independent policy families with stochastic replies; claim only results that hold across families; human pilot reported separately
MUS/MCS enumeration is slow on large instances	Restrict to the affected sub-problem; cap the number of MCS enumerated
Fine-tuned compiler underperforms the prompted model	Report the negative result; keep the prompted compiler; RQ3 still measures routing
The lightweight router is too inaccurate on unseen wording	Lower the share routed to the fast path by raising τ ; optionally swap in an open System-One model [27, 28] behind the same interface
Explanations hard to read	Group conflicts by stakeholder; evaluate readability with human raters
Portal or email integration takes too long	Keep the portal minimal; simulate all channels; research results do not depend on the UI
Free-tier rate limits or quota changes	Core results already run locally (Tier 1); shrink or drop the Tier 2 API validation

Table 14: Risks and fallbacks.

18 Open-Source Release

We will release under MIT (code) and CC-BY (our data): the request corpus with target constraints, the stakeholder simulators and negotiation scenarios with oracle outcomes, the CP-SAT model with conflict extraction, the LoRA adapters, the evaluation harness and all run logs, with a locked uv environment for one-command reproduction.

References

- [1] McCollum, B., et al. (2010). Setting the research agenda in automated timetabling: The Second International Timetabling Competition. *INFORMS Journal on Computing*, 22(1).
- [2] Di Gaspero, L., McCollum, B., Schaerf, A. (2007). The Second International Timetabling Competition (ITC-2007): Curriculum-based course timetabling (Track 3). In *Proceedings of the 14th RCRA Workshop on Experimental Evaluation of Algorithms for Solving Problems with Combinatorial Explosion*.
- [3] Google OR-Tools. CP-SAT Solver. Google for Developers (online documentation). https://developers.google.com/optimization/cp/cp_solver
- [4] Junker, U. (2004). QuickXplain: Preferred explanations and relaxations for over-constrained problems. *AAAI 2004*.

- [5] Liffiton, M.H., Sakallah, K.A. (2008). Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1).
- [6] Ramamonjison, R., et al. (2022). NL4Opt competition: Formulating optimization problems based on their natural language descriptions. *NeurIPS 2022 Competition Track*.
- [7] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*.
- [8] Dettmers, T., et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS 2023*.
- [9] Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- [10] Müller, T., Rudová, H., Barták, R. (2005). Minimal perturbation problem in course timetabling. In E.K. Burke, M.A. Trick (Eds.), *Practice and Theory of Automated Timetabling V*, LNCS 3616, 126–146. Springer. https://doi.org/10.1007/11593577_8
- [11] Phillips, A.E., Walker, C.G., Ehr Gott, M., Ryan, D.M. (2017). Integer programming for minimal perturbation problems in university course timetabling. *Annals of Operations Research*, 252(2), 283–304. <https://doi.org/10.1007/s10479-015-2094-z>
- [12] AhmadiTeshnizi, A., Gao, W., Udell, M. (2024). OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. *ICML 2024*.
- [13] Xiao, Z., et al. (2024). Chain-of-Experts: When LLMs meet complex operations research problems. *ICLR 2024*.
- [14] Tsouros, D., Verhaeghe, H., Kadioğlu, S., Guns, T. (2023). Holy Grail 2.0: From natural language to constraint models. PTHG 2023 Workshop at CP 2023. arXiv:2308.01589.
- [15] Xie, J., et al. (2024). TravelPlanner: A benchmark for real-world planning with language agents. *ICML 2024*.
- [16] Kambhampati, S., et al. (2024). Position: LLMs can’t plan, but can help planning in LLM-Modulo frameworks. *ICML 2024*.
- [17] Meignan, D., Knust, S., Frayret, J.-M., Pesant, G., Gaud, N. (2015). A review and taxonomy of interactive optimization methods in operations research. *ACM Transactions on Interactive Intelligent Systems*, 5(3), 1–43. <https://doi.org/10.1145/2808234>
- [18] Freuder, E.C. (2024). Conversational modeling for constraint satisfaction. *AAAI 2024*.
- [19] Lawless, C., et al. (2024). “I want it that way”: Enabling interactive decision support using large language models and constraint programming (the MeetMate system). *ACM Transactions on Interactive Intelligent Systems*, 14(3), Article 22. <https://doi.org/10.1145/3685053>
- [20] Marques-Silva, J., Heras, F., Janota, M., Previti, A., Belov, A. (2013). On computing minimal correction subsets. *IJCAI 2013*.
- [21] Gamba, E., Bogaerts, B., Guns, T. (2021). Efficiently explaining CSPs with unsatisfiable subset optimization. *IJCAI 2021*.
- [22] Gupta, S.D., Genc, B., O’Sullivan, B. (2021). Explanation in constraint satisfaction: A survey. *IJCAI 2021 (Survey Track)*.
- [23] Chen, H., Constante-Flores, G.E., Li, C. (2024). Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*.
- [24] Lewis, M., Yarats, D., Dauphin, Y., Parikh, D., Batra, D. (2017). Deal or no deal? End-to-end learning for negotiation dialogues. *EMNLP 2017*.
- [25] Abdelnabi, S., Gomaa, A., Sivaprasad, S., Schönherr, L., Fritz, M. (2024). Cooperation, competition, and maliciousness: LLM-stakeholders interactive negotiation. *NeurIPS 2024 Datasets and Benchmarks Track*.
- [26] Bertsimas, D., Farias, V.F., Trichakis, N. (2011). The price of fairness. *Operations Research*, 59(1).
- [27] TheoLeeCJ. SemIf-OpenJev: Semantic ifs from open models (optional alternative router). GitHub. <https://github.com/TheoLeeCJ/SemIf-OpenJev>
- [28] OpenJev Verdict (optional alternative router). <https://github.com/Heman10x-NGU/Verdict-open-jev>